

Day1 题解

Harry27182

目录

① 基础线段树练习题

② 基础 DP 练习题

③ 基础字符串练习题

① 基础线段树练习题

② 基础 DP 练习题

③ 基础字符串练习题

Statements

给定一个长度为 n 的序列 a 和一个模数 p ，你需要进行以下三种操作共 m 次：

- $1\ l\ r\ k$ ，表示区间 $[l, r]$ 内每个数加 k 。
- $2\ l\ r$ ，表示区间 $[l, r]$ 内每个数对 p 取模。
- $3\ l\ r$ ，查询区间 $[l, r]$ 内所有数的和。

Solutions

- 题目名字是线段树那么做法肯定不是线段树喵!

Solutions

- 题目名字是线段树那么做法肯定不是线段树喵!
- 考虑分块, 对于整块的修改, 可以简单利用 tag 维护。

Solutions

- 题目名字是线段树那么做法肯定不是线段树喵!
- 考虑分块, 对于整块的修改, 可以简单利用 tag 维护。
- 具体来说, 我们对每个块维护两个 tag, 分别表示最后一次取模前的加法和取模后的加法。

Solutions

- 题目名字是线段树那么做法肯定不是线段树喵!
- 考虑分块, 对于整块的修改, 可以简单利用 tag 维护。
- 具体来说, 我们对每个块维护两个 tag, 分别表示最后一次取模前的加法和取模后的加法。
- 考虑我们查询时需要哪些信息, 对于整块的查询, 我们最好维护出 $a_i \bmod p$ 排序后的结果, 这样之后前面的一段加上的是 $x \times p$, 后面的一段加上的是 $(x - 1) \bmod p$, 这个分界点可以二分出来。

Solutions

- 题目名字是线段树那么做法肯定不是线段树喵!
- 考虑分块, 对于整块的修改, 可以简单利用 tag 维护。
- 具体来说, 我们对每个块维护两个 tag, 分别表示最后一次取模前的加法和取模后的加法。
- 考虑我们查询时需要哪些信息, 对于整块的查询, 我们最好维护出 $a_i \bmod p$ 排序后的结果, 这样之后前面的一段加上的是 $x \times p$, 后面的一段加上的是 $(x - 1) \bmod p$, 这个分界点可以二分出来。
- 所以, 我们每次散块重构需要的也是这个信息, 也就是每次需要排序。

Solutions

- 题目名字是线段树那么做法肯定不是线段树喵!
- 考虑分块, 对于整块的修改, 可以简单利用 tag 维护。
- 具体来说, 我们对每个块维护两个 tag, 分别表示最后一次取模前的加法和取模后的加法。
- 考虑我们查询时需要哪些信息, 对于整块的查询, 我们最好维护出 $a_i \bmod p$ 排序后的结果, 这样之后前面的一段加上的是 $x \times p$, 后面的一段加上的是 $(x - 1) \bmod p$, 这个分界点可以二分出来。
- 所以, 我们每次散块重构需要的也是这个信息, 也就是每次需要排序。
- 平衡可以得到复杂度 $O(m\sqrt{n} \log n)$ 。

Solutions

- 如果你非常会卡常也许上面的做法可以通过，但正常情况下是不可以的。

Solutions

- 如果你非常会卡常也许上面的做法可以通过，但正常情况下是不可以的。
- 二分的 $\log$ 看上去就很难优化，考虑重构的时候可以依赖之前的信息进行优化。

Solutions

- 如果你非常会卡常也许上面的做法可以通过，但正常情况下是不可以的。
- 二分的 $\log$ 看上去就很难优化，考虑重构的时候可以依赖之前的信息进行优化。
- 一个块重构的时候内部元素只有两种不同的 tag 情况，这两种变换后的值循环移位之后可以变得有序。

Solutions

- 如果你非常会卡常也许上面的做法可以通过，但正常情况下是不可以的。
- 二分的 $\log$ 看上去就很难优化，考虑重构的时候可以依赖之前的信息进行优化。
- 一个块重构的时候内部元素只有两种不同的 tag 情况，这两种变换后的值循环移位之后可以变得有序。
- 所以可以把这两部分分开进行归并排序，复杂度 $O(m\sqrt{n\log n})$ ，在常数正常的情况下可以通过。

3 基础字符串练习题

Statements

小 P 有 n 个任务需要完成，这 n 个任务按照截止时间顺序从小到大给出，完成了第 i 个任务可以获得 w_i 的收益。

但小 P 有强迫症，TA 每次只会选择截止时间最小或第三小的任务去做。同时，由于小 P 的精力有限，所以有些任务 TA 不能连续去做。形式化的说，对于任务 i 和任务 j ，会给定 $s_{i,j}$ 表示能否在做完任务 i 之后下一个去做任务 j 。特别的，第一个做的任务是没有限制的。

小 P 想让自己的收益最大化，请你求出这个最大收益。

Solutions

- 容易写出一个简单的 $O(n^3)$ dp。下面不妨设 $i < j < k$ 。

Solutions

- 容易写出一个简单的 $O(n^3)$ dp。下面不妨设 $i < j < k$ 。
- 令 $f_{i,j,k}$ 表示上一个删的是第 i 个，且它是作为第一个被删掉的， j, k 分别为剩下的第一个和第二个的答案。

Solutions

- 容易写出一个简单的 $O(n^3)$ dp。下面不妨设 $i < j < k$ 。
- 令 $f_{i,j,k}$ 表示上一个删的是第 i 个，且它是作为第一个被删掉的， j, k 分别为剩下的第一个和第二个的答案。
- $g_{i,j,k}$ 表示上一个删的是 k ，且它是作为第三个被删掉的， i, j 分别为剩下的第二个和第三个的答案。

Solutions

- 容易写出一个简单的 $O(n^3)$ dp。下面不妨设 $i < j < k$ 。
- 令 $f_{i,j,k}$ 表示上一个删的是第 i 个，且它是作为第一个被删掉的， j, k 分别为剩下的第一个和第二个的答案。
- $g_{i,j,k}$ 表示上一个删的是 k ，且它是作为第三个被删掉的， i, j 分别为剩下的第二个和第三个的答案。
- 转移是显然的，复杂度 $O(n^3)$ 。

Solutions

- 优化 dp 首先需要压缩状态，我们再次仔细审视这个转移。

Solutions

- 优化 dp 首先需要压缩状态，我们再次仔细审视这个转移。
- $f_{i,j,k}$ 的两个转移方向是 $f_{j,k,k+1}$ 和 $g_{j,k,k+1}$ ，这让我们注意到我们其实只需要关注删完 i 之后是否可以删除 j 或 $k+1$ 即可。

Solutions

- 优化 dp 首先需要压缩状态，我们再次仔细审视这个转移。
- $f_{i,j,k}$ 的两个转移方向是 $f_{j,k,k+1}$ 和 $g_{j,k,k+1}$ ，这让我们注意到我们其实只需要关注删完 i 之后是否可以删除 j 或 $k+1$ 即可。
- 所以我们的 $f_{i,j,k}$ 状态其实可以压缩为 $f_{j,k,0/1,0/1}$ ，两个 0/1 分别表示能否转移到 $j, k+1$ 。

Solutions

- 优化 dp 首先需要压缩状态，我们再次仔细审视这个转移。
- $f_{i,j,k}$ 的两个转移方向是 $f_{j,k,k+1}$ 和 $g_{j,k,k+1}$ ，这让我们注意到我们其实只需要关注删完 i 之后是否可以删除 j 或 $k+1$ 即可。
- 所以我们的 $f_{i,j,k}$ 状态其实可以压缩为 $f_{j,k,0/1,0/1}$ ，两个 0/1 分别表示能否转移到 $j, k+1$ 。
- 但同时，我们还需要压缩 g 的状态，为了配合这里是转移到 $g_{j,k,k+1}$ ，我们直接令 $g_{j,k}$ 表示之前的 $g_{j,k,k+1}$ ，这样就可以快速处理 f 的转移。

Solutions

- 优化 dp 首先需要压缩状态，我们再次仔细审视这个转移。
- $f_{i,j,k}$ 的两个转移方向是 $f_{j,k,k+1}$ 和 $g_{j,k,k+1}$ ，这让我们注意到我们其实只需要关注删完 i 之后是否可以删除 j 或 $k+1$ 即可。
- 所以我们的 $f_{i,j,k}$ 状态其实可以压缩为 $f_{j,k,0/1,0/1}$ ，两个 0/1 分别表示能否转移到 $j, k+1$ 。
- 但同时，我们还需要压缩 g 的状态，为了配合这里是转移到 $g_{j,k,k+1}$ ，我们直接令 $g_{j,k}$ 表示之前的 $g_{j,k,k+1}$ ，这样就可以快速处理 f 的转移。
- 下面只需要考虑如何快速处理 g 的转移。

Solutions

- 我们考虑压缩转移路径，我们发现 $g_{i,j,j+1}$ 的转移路径为 $g_{i,j,j+1} \rightarrow g_{i,j,p-1} \rightarrow f_{i,j,p}$ 。

Solutions

- 我们考虑压缩转移路径，我们发现 $g_{i,j,j+1}$ 的转移路径为 $g_{i,j,j+1} \rightarrow g_{i,j,p-1} \rightarrow f_{i,j,p}$ 。
- 我们不再考虑 $g \rightarrow g$ 的转移，钦定 f 为主 dp 数组， g 为辅助数组，直接考虑 g 第一次转移到 f 的结果。

Solutions

- 我们考虑压缩转移路径，我们发现 $g_{i,j,j+1}$ 的转移路径为 $g_{i,j,j+1} \rightarrow g_{i,j,p-1} \rightarrow f_{i,j,p}$ 。
- 我们不再考虑 $g \rightarrow g$ 的转移，钦定 f 为主 dp 数组， g 为辅助数组，直接考虑 g 第一次转移到 f 的结果。
- 枚举 j ，容易注意到 i 和 p 部分的贡献是独立的，所以可以先按照 i 部分贡献从大到小排序。

Solutions

- 我们考虑压缩转移路径，我们发现 $g_{i,j,j+1}$ 的转移路径为 $g_{i,j,j+1} \rightarrow g_{i,j,p-1} \rightarrow f_{i,j,p}$ 。
- 我们不再考虑 $g \rightarrow g$ 的转移，钦定 f 为主 dp 数组， g 为辅助数组，直接考虑 g 第一次转移到 f 的结果。
- 枚举 j ，容易注意到 i 和 p 部分的贡献是独立的，所以可以先按照 i 部分贡献从大到小排序。
- 用 bitset 维护没有更新过的位置，每次找到当前能更新的位置（一些 bitset 的交），不断 findnext 更新即可。

Solutions

- 我们考虑压缩转移路径，我们发现 $g_{i,j,j+1}$ 的转移路径为 $g_{i,j,j+1} \rightarrow g_{i,j,p-1} \rightarrow f_{i,j,p}$ 。
- 我们不再考虑 $g \rightarrow g$ 的转移，钦定 f 为主 dp 数组， g 为辅助数组，直接考虑 g 第一次转移到 f 的结果。
- 枚举 j ，容易注意到 i 和 p 部分的贡献是独立的，所以可以先按照 i 部分贡献从大到小排序。
- 用 bitset 维护没有更新过的位置，每次找到当前能更新的位置（一些 bitset 的交），不断 findnext 更新即可。
- 复杂度 $O(\frac{n^3}{w})$ ，可以通过。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

Statements

给定小写字母组成的字符串 S 以及 S 的两个非空子串 A, B 。记 $L(A)$ 表示 A 在 S 中出现位置的左端点。称 B 是 A 的强连续当且仅当 A 每次出现后都有一个 B 出现 (即 $L(A) = L(AB)$)。称 B 是 A 的弱连续当且仅当 A 每次出现后都有 B 出现, 或者 A 出现后的字符数不足 $|B|$ 个, 且至少一个 B 在 A 后出现。

如在字符串 `abacaba` 中, `a` 是 `ab` 的连续, `bacab` 是 `a` 的弱连续, 但 `ba` 就不是 `a` 的弱连续 (因为在其中一个 `a` 出现后, 紧接着的是 `ca` 而不是 `ba`)。

现在小 P 拿到了字符串 S , 他想知道有多少 S 的非空子串二元组 (A, B) 满足 B 是 A 的弱连续。

Solutions

- 考虑建出反串的 SAM，考虑枚举串 AB 在 fail 树上的节点 x ，那么我们知道 A 串节点 y 在 x 的祖先上。

Solutions

- 考虑建出反串的 SAM, 考虑枚举串 AB 在 fail 树上的节点 x , 那么我们知道 A 串的节点 y 在 x 的祖先上。
- 记一个节点 x 的代表的 stapos 集合为 S_x 。那么考虑 $p = \min_{u \in S_y/S_x} u$, 则需要满足 $|AB| > n - p + 1$ 。

Solutions

- 考虑建出反串的 SAM, 考虑枚举串 AB 在 fail 树上的节点 x , 那么我们知道 A 串的节点 y 在 x 的祖先上。
- 记一个节点 x 的代表的 stapos 集合为 S_x 。那么考虑 $p = \min_{u \in S_y/S_x} u$, 则需要满足 $|AB| > n - p + 1$ 。
- 于是可以注意到必须有 $\min_{u \in S_y} u = \min_{u \in S_x} u$, 否则一定不合法。感性理解就是如果 A 第一次出现的时候 AB 就不出现那么肯定不合法。

Solutions

- 考虑建出反串的 SAM, 考虑枚举串 AB 在 fail 树上的节点 x , 那么我们知道 A 串的节点 y 在 x 的祖先上。
- 记一个节点 x 的代表 stapos 集合为 S_x 。那么考虑 $p = \min_{u \in S_y/S_x} u$, 则需要满足 $|AB| > n - p + 1$ 。
- 于是可以注意到必须有 $\min_{u \in S_y} u = \min_{u \in S_x} u$, 否则一定不合法。感性理解就是如果 A 第一次出现的时候 AB 就不出现那么肯定不合法。
- 而在 parent 树上, y 的每个儿子对应的 $\min_{u \in S_{son}} u$ 应该都不同, 这启示我们按照 $\min_{u \in S_x} u$ 的最小值进行重链剖分。

Solutions

- 具体来说，我们钦定 $\min_{u \in S_x} u$ 最小的儿子为重儿子，只需要考虑 x, y 在同一条重链上的情况。

Solutions

- 具体来说，我们钦定 $\min_{u \in S_x} u$ 最小的儿子为重儿子，只需要考虑 x, y 在同一条重链上的情况。
- 这样我们就把树上问题转化为了链上问题。

Solutions

- 具体来说，我们钦定 $\min_{u \in S_x} u$ 最小的儿子为重儿子，只需要考虑 x, y 在同一条重链上的情况。
- 这样我们就把树上问题转化为了链上问题。
- 从链顶到链底做扫描线，容易发现答案可以表示为 $((n - p_y + 1, n] \cap (len_{fa_x}, len_x]) \times (len_y - len_{fa_y})$ ，其中 p_y 表示扫到当前的 x 对应的 $\min_{u \in S_y/S_x} u$ 。

Solutions

- 具体来说，我们钦定 $\min_{u \in S_x} u$ 最小的儿子为重儿子，只需要考虑 x, y 在同一条重链上的情况。
- 这样我们就把树上问题转化为了链上问题。
- 从链顶到链底做扫描线，容易发现答案可以表示为 $((n - p_y + 1, n] \cap (len_{fa_x}, len_x]) \times (len_y - len_{fa_y})$ ，其中 p_y 表示扫到当前的 x 对应的 $\min_{u \in S_y/S_x} u$ 。
- p_y 需要使用单调栈进行维护，所以扫描线时用线段树 + 单调栈维护答案即可。

Solutions

- 具体来说，我们钦定 $\min_{u \in S_x} u$ 最小的儿子为重儿子，只需要考虑 x, y 在同一条重链上的情况。
- 这样我们就把树上问题转化为了链上问题。
- 从链顶到链底做扫描线，容易发现答案可以表示为 $((n - p_y + 1, n] \cap (len_{fa_x}, len_x]) \times (len_y - len_{fa_y})$ ，其中 p_y 表示扫到当前的 x 对应的 $\min_{u \in S_y/S_x} u$ 。
- p_y 需要使用单调栈进行维护，所以扫描线时用线段树 + 单调栈维护答案即可。
- 复杂度 $O(n \log n)$ ，可以通过。

Solutions

- 具体来说，我们钦定 $\min_{u \in S_x} u$ 最小的儿子为重儿子，只需要考虑 x, y 在同一条重链上的情况。
- 这样我们就把树上问题转化为了链上问题。
- 从链顶到链底做扫描线，容易发现答案可以表示为 $((n - p_y + 1, n] \cap (len_{fa_x}, len_x]) \times (len_y - len_{fa_y})$ ，其中 p_y 表示扫到当前的 x 对应的 $\min_{u \in S_y / S_x} u$ 。
- p_y 需要使用单调栈进行维护，所以扫描线时用线段树 + 单调栈维护答案即可。
- 复杂度 $O(n \log n)$ ，可以通过。
- 本题也存在 SA 的做法，但由于本题的结构天然适合 stapos 集合更适配 SAM，SAM 的想法也就更自然，所以在此就不再赘述了。